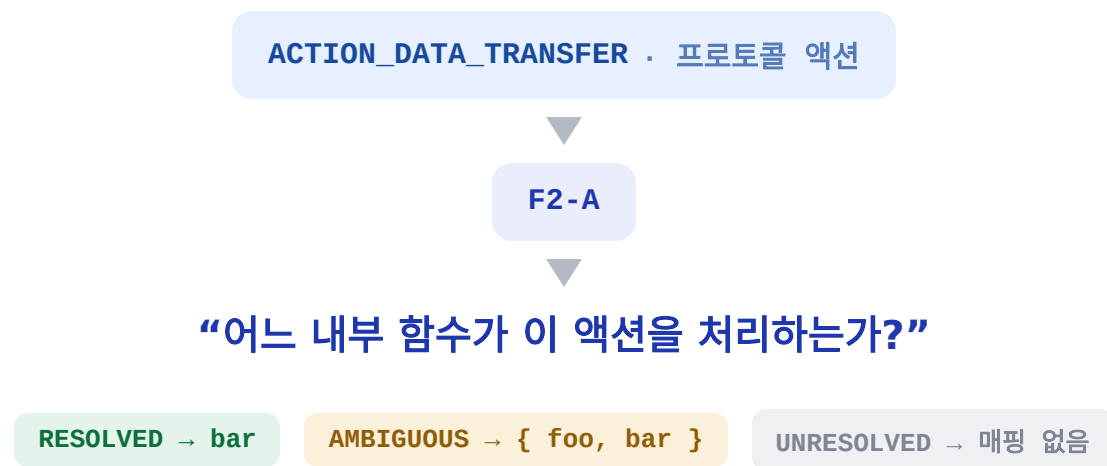


F2-A는 하나의 질문에 답한다



이 질문이 왜 중요한가

보안 질문 — “chargingLimit 은 검증되는가?”



필요로 함

핸들러 진입 함수



가능케 함

필드 바인딩

오염 소스

데이터 흐름

싱크·검사

핸들러가 없으면 하위 프로토콜 분석에 신뢰할 출발점이 없다

입력과 출력 — 인터페이스부터

입력 1 · KB 액션 정의

canonical name · symbols · numeric IDs · aliases

입력 2 · 프로그램 모델

AST / CPG · 호출 · 대입 · 초기화 · 심볼 해석



F2-A

출력 · 액션당

status · **chosen**(확정 시) · ranked candidates · evidence · conflict · unresolved reason

아직 아키텍처가 아니라 **인터페이스**를 정의한다

F2-A가 하지 않는 것

F2-A 한다

- 액션 → 핸들러 해석
- 후보 수집 · 증거 보존
- 신뢰도 · 순위
- 모호 · 실패 보고

F2-A 안 한다

- 페이로드 필드 추적
- 오염 경로 계산
- 싱크 분류 · 검사 검증
- 최종 취약점 판정

F2-A 출력



하위 F3 · F4 · F5 · F6

이제 실행할 여섯 단계

- 1 **액션 하나**를 잡는다.
- 2 그 액션을 **인코딩한 코드 위치**를 찾는다.
- 3 그 위치들의 **콜백을 모두** 뽑는다.
- 4 관찰을 **증거**로 바꾼다.
- 5 콜백별로 **점수 매겨 비교**한다.
- 6 **확정·모호·실패** 중 하나를 낸다.

다음 → 이 여섯 단계를 **ACTION_DATA_TRANSFER** 하나로 끝까지 따라간다

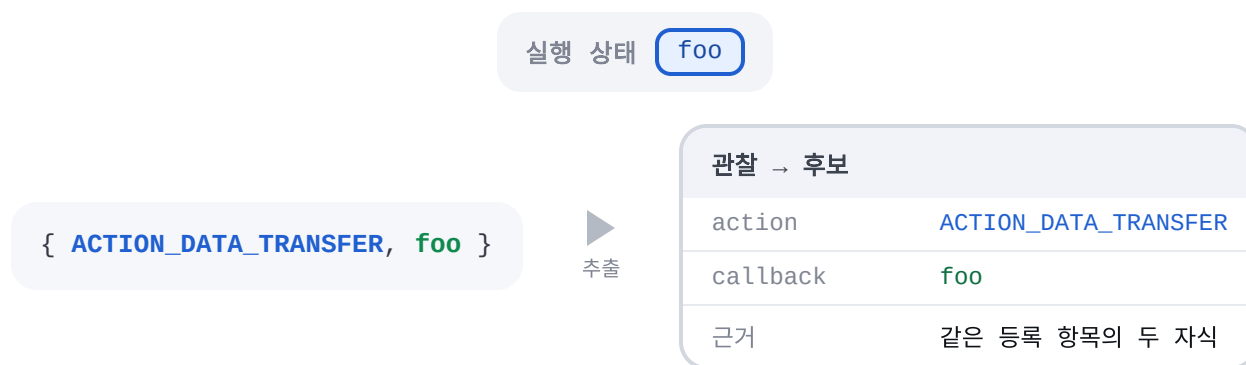
우리가 실행할 프로그램

실행 상태 후보 없음

```
static Reg table[] = {  
    { ACTION_DATA_TRANSFER, foo } // 등록  
};  
void dispatch(Frame *f) {  
    switch (f->action) {  
        case ACTION_DATA_TRANSFER:  
            bar(f->payload); // 디스패치  
    }  
}
```

다음 → 이 액션의 핸들러는? 먼저 코드에서 후보를 찾자.

1단계 등록부에서 첫 후보 **foo** 를 찾는다



다음 → 그런데 등록은 한 가지 형태로만 쓰이지 않는다.

같은 등록, 여러 표기 — 모두 같은 **foo** 로



묶는 조건 — 같은 슬롯일 때만

실행 상태 foo

허용 — 같은 슬롯 k

```
t[k].action = ACTION_DATA_TRANSFER;
t[k].fn      = foo;
```

▶ ACTION... → foo

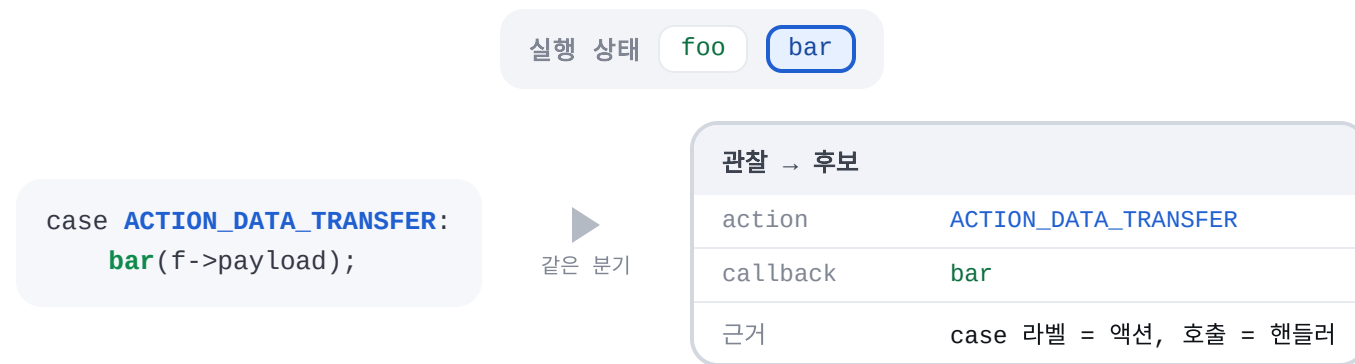
거부 — 다른 인덱스

```
t[k].action = ACTION_DATA_TRANSFER;
t[m].fn      = foo;
```

▶ k==m 증명 불가 → 묶지 않음

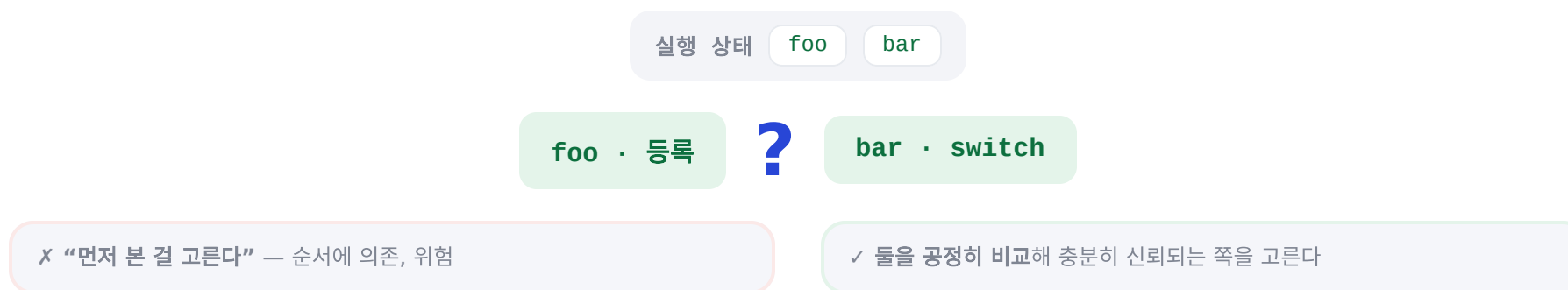
다음 → 등록 말고 다른 곳에도 이 액션의 후보가 있을 수 있다.

2단계 switch 에서 둘째 후보 bar 를 찾는다



다음 → 이제 후보가 둘이다 — 어느 것이 진짜 핸들러일까?

후보가 둘 — 첫 번째를 그냥 고를 순 없다



다음 → 공정히 비교하려면, 두 관찰을 같은 형식으로 만들어야 한다.

3단계 두 관찰을 같은 형식의 증거로 바꾼다

실행 상태

foo · EvA

bar · EvB

Evidence A

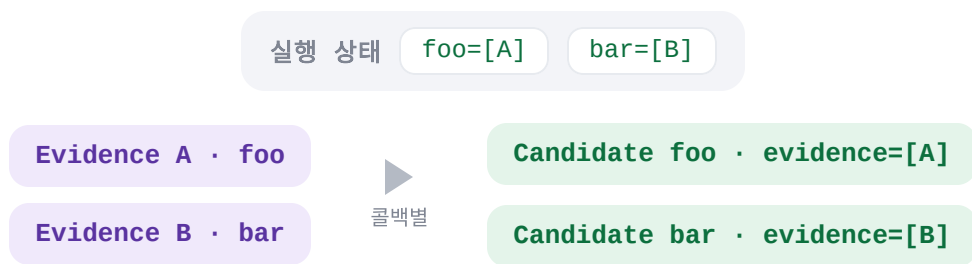
kind	REGISTRATION_INIT
callback	foo
match	HEURISTIC_SUBSTRING
provenance	등록 항목 #0

Evidence B

kind	ENUM_CASE
callback	bar
match	NORMALIZED_NAME
provenance	switch 분기

다음 → 두 증거를 무엇을 기준으로 비교하지?

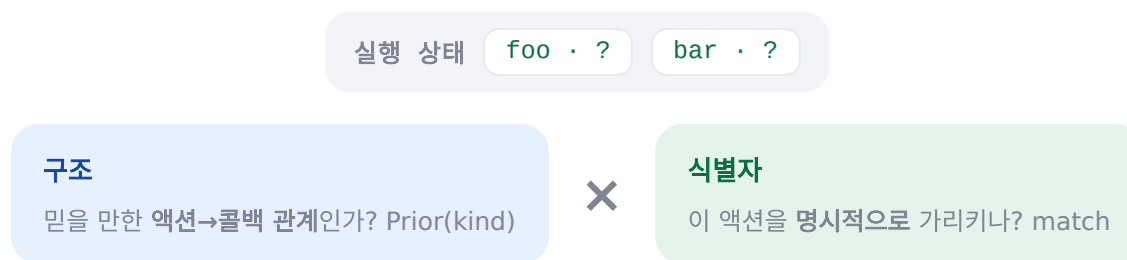
4단계 증거를 콜백별 후보로 묶는다



비교 단위는 증거가 아니라 **콜백 후보** — 각 후보가 자기 증거를 들고 있다

다음 → 이제 후보별로 점수를 매기자 — 점수는 무엇으로 정하나?

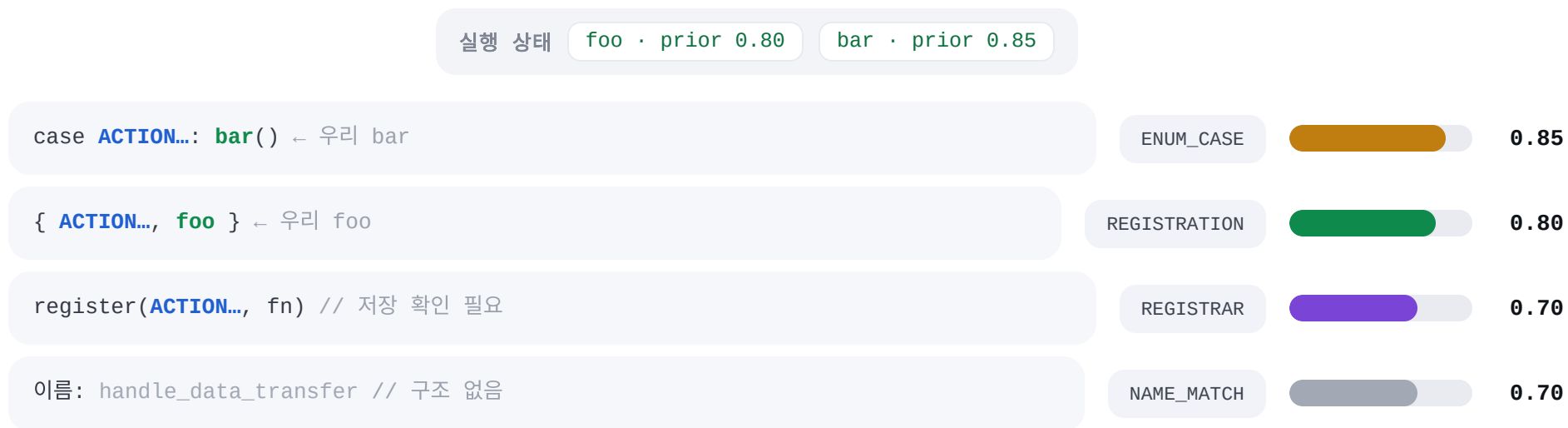
5단계 점수 = 구조 신뢰 × 식별자 대응



둘 중 하나라도 약하면 전체 증거도 제한된다

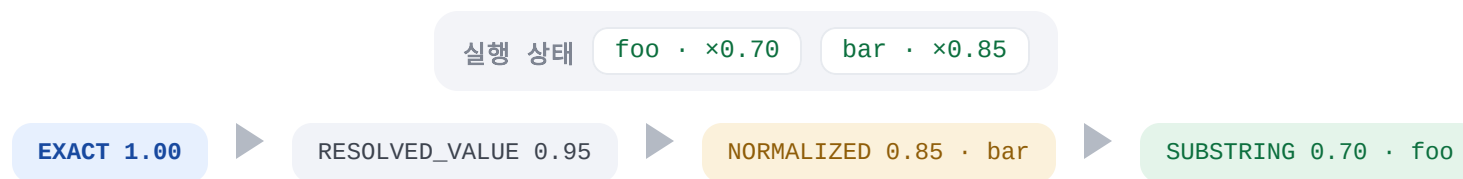
다음 → 먼저 “구조 신뢰”는 어디서 오나?

구조 신뢰 — 관계를 얼마나 직접 표현하나



다음 → [두 번째 재료 — 식별자 대응은 얼마나 강한가?](#)

식별자 대응 — 명시인가, 닮은 것뿐인가



우리 bar 는 정규화 일치(0.85), foo 는 부분 문자열(0.70) — 식별자 대응은 **bar** 가 더 강하다

다음 → 이제 **foo**부터 실제 점수를 계산하자.

후보 **foo** — 같은 등록의 관찰 셋을 한 번만 센다

실행 상태

foo · 0.56

bar · ?

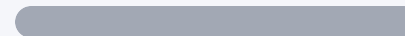
같은 등록에서 관찰이 셋

table-entry 0.80 · handler-ref 0.80 · symbol 0.80

다 더하면? 한 등록이 세 증인처럼 과장

같은 출처는 한 번만 → **MAX**

MAX = 0.80 × match 0.70



0.56

provenance grouping

다음 → 이제 **bar**도 같은 방식으로 계산하자.

후보 **bar** — $0.85 \times 0.85 = 0.72$

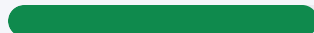
실행 상태

foo · 0.56

bar · 0.72

bar · ENUM_CASE

prior 0.85 × match NORMALIZED 0.85



0.72

foo · 0.56

VS

bar · 0.72

다음 → 근거가 더 있었다면 점수가 달라졌을까?

만약 한 후보가 다른 곳에서도 확인된다면?



만약 근거가 약한 이름뿐이라면?

실행 상태

foo · 0.56

bar · 0.72

정확 매치 있음 → cap 미적용

약한 이름 3개 결합



0.973

약한 근거가 구조적 증명처럼 과신?

정확 식별자 증거가 없으면 → cap



0.85

아무리 강해도 global cap 0.99 미만

우리 실행의 bar 는 정규화·정확 대응이 있어 **이 상한에 걸리지 않는다**

다음 → **이제 확정 점수로 돌아가 — bar 0.72 vs foo 0.56, 결정하자.**

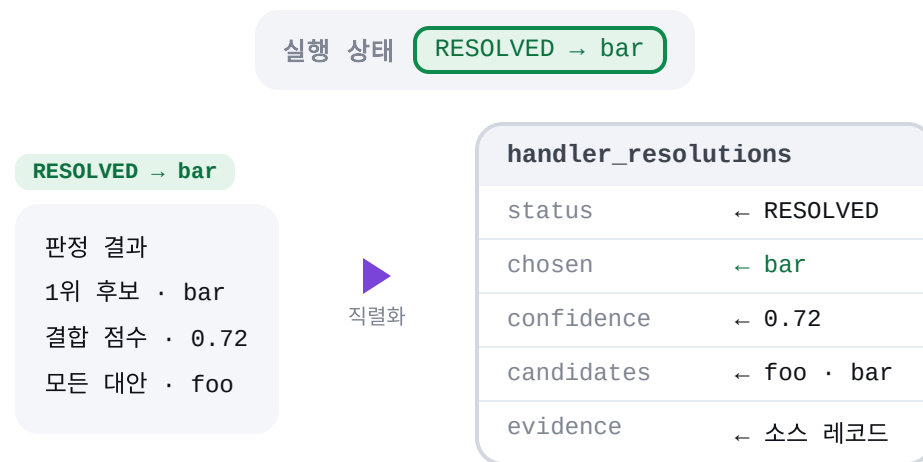
6단계 두 후보를 비교한다



6단계 두 질문을 통과 → bar 로 확정



판정을 handler_resolutions 로 내보낸다



다음 → 이 출력을 누가 쓰나?

확정된 **bar** 가 오염 분석의 출발점이 된다

실행 상태

chosen = bar

확정 핸들러 · BAR

```
int bar(Payload *p) {
  int limit = p->chargingLimit;
  set_current_limit(limit);
}
```

필드 chargingLimit

변수 limit

싱크 set_current_limit

만약 UNRESOLVED 였다면

미해결

신뢰할 소스 루트 없음

이 액션의 하위 분석 중단

다음 → 그렇다면 **F2-A**가 못 푸는 경우는 언제인가?

정적으로 슬롯을 증명할 수 있는가가 경계다

지원 — 같은 슬롯

```
t[k].action = ACTION...;
t[k].fn      = foo;
```

같은 수신자 + 같은 슬롯 → 복원 가능

미지원 — 런타임 탐색

```
for (...) if (t[i].action==action)
    t[i].fn = foo;
```

런타임 탐색 + 심볼릭 인덱스 → **미해결 · 사유 명명**

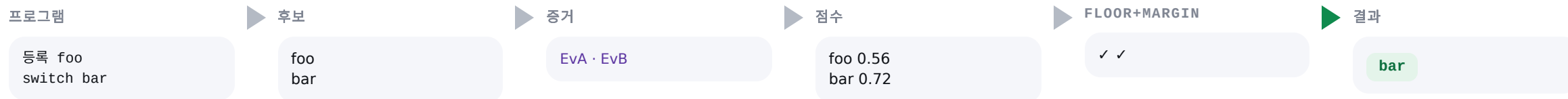
다음 → 이 동작을 무엇으로 보장하나?

테스트는 '구현 동작'을 고정하지, '현실 정확도'를 증명하진 않는다



고정 픽스처 테스트 = 구현 동작 증명 · 임의 펌웨어의 현실 정확도는 별개

우리가 방금 실행한 전체 트레이스



단계마다 어떤 객체가 있었고, 무엇이 바뀌었는지를 하나의 예제로 따라왔다

부록 — 상수 · 사유 · 결합 방식 비교

사전확률 · 매치 배수 · 임계값

STRING_DISPATCH 0.90 · ENUM_CASE 0.85 REGISTRATION_* 0.80 · REGISTRAR
0.70 · NAME 0.70/0.65 EXACT 1.00 · RESOLVED_VALUE 0.95 · NORMALIZED
0.85 SUBSTRING 0.70 · NONE 0.00 floor 0.50 · margin 0.15 · weak-only
0.85 · global 0.99 · depth 2

미해결 사유 · 결합 (0.80·0.49)

NO_EVIDENCE · LOW_CONFIDENCE · EXTERNAL_DEFINITION
REGISTRAR_STORE_NOT_REACHED · SEARCH_THEN_WRITE MISSING_POINTSTO ·
DYNAMIC_ACTION_ID · OUT_OF_TU — MAX 0.80 · SUM 1.29 · AVG 0.645 ·
[NOISY-OR 0.898](#)

본문은 하나의 실행 트레이스에 집중 · 수치·전체 분류는 여기에만